

Presentation Speaker Script

Intelligent Platform for Supervision and Automated Security

Design and deployment of a centralized supervision and security platform with artificial intelligence and automated response

Purpose: use this document as parallel speaker notes. It follows the presentation slide by slide and explains the logic, components, process, and project value.

Prepared for: Ghazi Mabrouki

ArcTic 3 | Huawei Enterprise Cloud Department - Innovation Department | 2026

How to Use This Script

This script is written to be spoken in parallel with the slides. The objective is not to read every word mechanically, but to follow the same structure while explaining what each component does, why it exists, and how the full SOC process works.

Recommended presentation style: introduce the problem, show the architecture, explain the three phases, then focus more deeply on Phase 3 because it contains the intelligence, correlation, AI analysis, approval, and remediation logic.

Main message to repeat: This is not only a dashboard. It is an end-to-end SOC workflow: detect, enrich, correlate, investigate, explain, approve, remediate, verify, and archive.

Table of Contents

Slide 1 - Title

Slide 2 - Table of Contents

Slides 3-5 - Introduction and Context

Slides 6-7 - Existing Solutions

Slides 8-9 - Architecture and Proposed Solution

Slides 10-13 - Phase 1: Monitoring Foundation

Slides 14-16 - Phase 2: Cloud Infrastructure

Slides 17-23 - Phase 3: Backend Intelligence Layer

Slides 24-26 - Functional and Non-Functional Requirements

Slides 27-28 - Testing Strategy

Slides 29-30 - Methodology

Slides 31-32 - Conclusion

Final Slide - Thank You

Final Checklist - Corrections before presenting

Opening

Slide 1 - Title

Good morning/afternoon everyone. Today I will present my project entitled **Intelligent Platform for Supervision and Automated Security**.

The main idea of this project is to design and deploy a centralized SOC platform capable of collecting security data, monitoring infrastructure, detecting threats, analyzing incidents with artificial intelligence, and supporting controlled automated remediation.

So the project is not only a monitoring dashboard. It is an end-to-end security workflow, from raw logs and alerts to incident investigation, AI analysis, Ansible playbook generation, approval, execution, and verification.

Slide 2 - Table of Contents

This presentation is structured into ten main parts.

I will first introduce the project context and the problems we are trying to solve. Then I will present the existing solutions and the gap identified. After that, I will explain the global architecture of the proposed solution.

The implementation is divided into three phases: the monitoring foundation, the cloud infrastructure, and the backend intelligence layer. Then I will present the functional and non-functional requirements, the testing strategy, the methodology followed during development, and finally the conclusion.

01 - Introduction and Context

Slide 3 - Section Introduction

In this first part, I will explain the context of the project, the main challenges in modern monitoring and security systems, and the objectives of the proposed platform.

Slide 4 - Context and Challenges

In many infrastructures, monitoring and security tools are deployed separately. For example, one tool can monitor system resources, another one detects network attacks, another one detects host intrusions, and another one analyzes runtime behavior.

The problem is that these tools are often fragmented. They generate heterogeneous data, with different formats and different levels of technical complexity. This makes it difficult for analysts to have a unified view of what is really happening.

Another challenge is expert dependency. Many alerts are highly technical, so the analyst has to manually understand, correlate, and interpret them. This slows down response time.

Finally, corrective actions are often manual. This means that even if an attack is detected, the reaction can be delayed. The result is slower incident response, missed multi-stage attacks, and analyst fatigue.

Here, present the problem clearly. Make the audience understand why the platform is necessary before explaining the solution.

Slide 5 - Project Objectives

The project has three main objectives.

The first objective is **complete visibility**. The platform must monitor several layers: network activity, host behavior, runtime events, and infrastructure metrics. This reduces blind spots.

The second objective is **intelligent processing**. Instead of keeping raw alerts isolated, the backend transforms them into normalized alerts, enriches them with context, and correlates them into incidents.

The third objective is **controlled response**. The system can generate investigations and remediation playbooks using AI, but it does not execute actions blindly. Every action goes through validation, approval, execution control, and audit.

So the logic is: observe everything, understand what is happening, and respond safely.

02 - Existing Solutions

Slide 6 - Section Introduction

In this part, I will briefly discuss existing monitoring and SOC solutions, and explain why a unified approach is needed.

Slide 7 - State of the Art of SOC Solutions

Existing SOC and SIEM solutions are powerful, but they often focus on specific parts of the workflow.

Some solutions are strong in log collection and visualization. Others are strong in intrusion detection, endpoint monitoring, or automation. But the gap I identified is that few solutions combine all these capabilities in one coherent architecture.

The goal of my project is to combine multi-layer monitoring, automatic enrichment, incident correlation, AI-based analysis, and automated remediation in a unified cloud-deployable platform.

So the contribution is not to replace existing tools, but to integrate them into one intelligent operational workflow.

Keep this slide short. The most important idea is the identified gap: existing tools are useful, but the workflow is not unified enough.

03 - Architecture and Proposed Solution

Slide 8 - Section Introduction

Now I will present the global architecture of the proposed solution and explain how the different components communicate with each other.

Slide 9 - Global Platform Architecture

This slide represents the global architecture of the platform.

The project is divided into three main phases.

Phase 1 is the monitoring foundation. It includes tools such as Suricata, Wazuh, Falco, Telegraf, Filebeat, Elasticsearch, and Kibana. The goal here is to collect, store, and visualize security and infrastructure events.

Phase 2 is the cloud infrastructure. The platform is deployed on Huawei Cloud using an ECS virtual machine, VPC, subnet, security groups, and an elastic IP.

Phase 3 is the backend intelligence layer. This is where raw alerts become normalized alerts, then correlated incidents, then investigations, AI analysis, playbooks, approvals, and remediation.

The global data flow is: security sources generate events, the collection layer sends them to Elasticsearch, Kibana visualizes them, the backend analyzes and enriches them, and the frontend dashboard presents alerts, incidents, metrics, and actions to the analyst.

When presenting this slide, move from left to right: sources, collection, storage, visualization, backend intelligence, then frontend dashboard.

04 - Phase 1: Monitoring Foundation

Slide 10 - Section Introduction

Phase 1 is the foundation of the whole project. Before adding AI or automation, the platform must first collect reliable security data from different layers of the infrastructure.

Slide 11 - Visibility Layers and Tools

In this phase, the objective is to create visibility across different layers.

At the network layer, **Suricata** is used as an IDS to detect suspicious network traffic, such as scans, brute force attempts, malware traffic, or known threat signatures.

At the host layer, **Wazuh** monitors system activity, authentication logs, file integrity, and host-based security events.

At the runtime layer, **Falco** detects suspicious behavior inside containers or Linux systems, such as unexpected shell execution or abnormal process activity.

At the infrastructure layer, **Telegraf** collects metrics like CPU, memory, disk, network usage, and system load.

All of these tools are connected to a centralized storage and visualization layer using Elasticsearch and Kibana.

Slide 12 - Phase 1 Architecture and Data Flow

This slide explains the technical data flow of Phase 1.

The sources are system logs, authentication logs, Suricata IDS alerts, Wazuh HIDS alerts, Falco runtime events, and Telegraf metrics.

Filebeat reads logs and forwards them to Elasticsearch. For example, Suricata writes alerts into `eve.json`, and Filebeat sends those alerts to Elasticsearch. Wazuh alerts are also collected and sent through the same logic.

For Falco, the alerts are received by **Falcosidekick**, which forwards them to Elasticsearch.

Telegraf sends system metrics directly to Elasticsearch.

Then **Elasticsearch** acts as the central storage, indexing, and search engine. Finally, **Kibana** is used to visualize dashboards, explore logs, and analyze events.

So the logic is: collect from multiple security sources, centralize in Elasticsearch, then visualize and analyze through Kibana.

Slide 13 - Phase 1 Deployment and Result

In terms of deployment, the SIEM core is kept stable and centralized.

Some tools are installed as native services, such as Elasticsearch, Kibana, Filebeat, Suricata, Wazuh, and Telegraf. Falco and Falcosidekick are deployed as Docker services because runtime detection can be isolated more easily in containers.

This architecture is simple, maintainable, and adapted to the project needs.

At the end of Phase 1, the result is a complete security monitoring foundation. The platform can monitor network activity, host behavior, runtime events, and system resources. All data is centralized in Elasticsearch and visualized in Kibana.

This phase prepares the platform for cloud deployment and backend intelligence.

05 - Phase 2: Cloud Infrastructure

Slide 14 - Section Introduction

After building the monitoring foundation, Phase 2 focuses on deploying the platform in a real cloud environment.

Slide 15 - Huawei Cloud Environment

In Phase 2, the platform is deployed on Huawei Cloud.

The infrastructure is based on an ECS virtual machine with strong resources: 64 vCPU, 128 GB RAM, and 100 GB storage. These resources are important because Elasticsearch, Kibana, Wazuh, and the backend intelligence layer can require significant computing and memory capacity.

The machine runs Ubuntu Server and hosts the complete SOC stack: Elasticsearch, Kibana, Wazuh, Suricata, Falco, Telegraf, Filebeat, and Docker.

The network architecture includes a private VPC, a private subnet, security groups, and an elastic public IP. The VPC isolates the environment, the subnet organizes internal communication, and security groups control which ports are accessible.

Slide 16 - Security Strategy and Integration

The security strategy is very important in this phase.

The main principle is that Elasticsearch must remain internal and must not be publicly exposed. Elasticsearch contains sensitive logs and security data, so exposing it directly would be dangerous.

Access to Kibana and SSH must be restricted to authorized sources only. This is controlled using Huawei Cloud security groups and firewall rules.

This cloud phase connects the theoretical architecture to real operations. It hosts the full SOC stack and creates the foundation for the backend intelligence layer.

So Phase 2 answers the question: where and how do we securely run the platform in a real environment?

Important correction: in the requirements slide, replace 'Elasticsearch expose publiquement' with 'Elasticsearch not publicly exposed' or 'Elasticsearch remains internal.'

06 - Phase 3: Backend Intelligence Layer

Slide 17 - Section Introduction

Phase 3 is the most intelligent part of the project. The goal here is to transform raw monitoring data into real SOC operations.

Slide 18 - Phase 3 Global Overview

This slide shows the complete intelligence flow.

The backend starts from raw data collected in Elasticsearch. Then it creates normalized alerts, correlates them into incidents, generates investigations, sends the context to the AI engine, produces playbooks, waits for approval, executes actions, verifies the result, and finally archives everything.

The 12 key steps are: ingestion and enrichment, alert transformation, contextual enrichment, local storage, incident correlation, investigation creation, AI response engine, approval workflow, execution and verification, performance monitoring, IPS visualization, and AI assistant or AI operator.

So Phase 3 is the bridge between detection and response. It transforms the platform from a passive monitoring system into an active SOC assistant.

Slide 19 - Step 1: Alert Ingestion and Enrichment

The first step of the backend intelligence layer is alert ingestion.

The backend reads security data from Elasticsearch indices such as wazuh-alerts-*, suricata-*, falco-events-*, filebeat-*, and telegraf-*.

Each source has its own format. For example, a Wazuh alert is different from a Suricata alert, and a Falco event is different from a Telegraf metric. That is why the backend uses mappers to transform source-specific data into a normalized alert model.

After that, the backend performs deduplication to avoid repeated alerts, filters noise, enriches the alert with GeoIP and MITRE information, checks whitelists, stores the result in SQLite, and broadcasts updates through WebSocket.

The result is a clean normalized alert containing source, severity, category, IPs, hostname, tags, indicators of compromise, MITRE techniques, metadata, and occurrence count.

Simple phrase: Here, the backend turns noisy raw alerts into clean and usable security alerts.

Slide 20 - Step 2: Incident Correlation

After alert normalization, the backend performs incident correlation.

The goal is to avoid treating every alert as an isolated event. In real attacks, several alerts can belong to the same campaign.

The correlation engine uses signals such as source IP, hostname, container ID, MITRE techniques, attack pattern, kill-chain progression, and time window.

For example, Suricata can detect a port scan from one IP address. Then Wazuh can detect SSH brute force from the same IP. After that, Falco can detect suspicious shell activity on the target system.

Individually, these are separate alerts. But together, they represent a stronger attack scenario. The backend correlates them and creates one incident with higher confidence, such as an SSH brute force and port scan campaign.

This is important because it gives the analyst context instead of alert overload.

This is one of the strongest technical slides. Explain the example clearly and slowly.

Slide 21 - Step 3: Investigation and AI Engine

When an incident is created, the backend can automatically create an investigation.

The investigation contains rich context: the full timeline of related alerts, involved IPs, ports, services, domains, MITRE tactics and techniques, authentication patterns, behavioral indicators, and a dynamic risk score.

Then the AI response engine uses this context to generate an investigation summary, a narrative of the attack chain, a risk evaluation, and a remediation playbook.

The supported AI providers include Ollama locally, Google Gemini, OpenRouter, and NVIDIA NIM.

However, the most important principle is that the AI does not directly execute actions. It assists the analyst by explaining the incident and proposing remediation, but the output must still pass validation and approval.

Slide 22 - Step 4: Approval, Execution and Verification

This slide explains the controlled response workflow.

There are different approval types. In manual approval, the analyst reviews the proposed action and decides whether to approve it or not. In low-risk cases, some actions can be auto-approved. But high-risk or dangerous actions can be blocked.

Before executing anything, the backend validates the YAML syntax of the Ansible playbook. It also checks SSH connectivity to the target, verifies whitelist protection, and logs the complete execution process.

After execution, the platform verifies the result. For example, if the action is to block an IP, the system must check that the rule was really applied.

So the backend does not execute blindly. It follows a safe workflow: generate, validate, approve, execute, verify, and archive.

Key phrase: The backend thinks before acting.

Slide 23 - Phase 3 Operational Modules

Phase 3 also includes several operational modules.

The first module is **performance monitoring**. It continuously reads metrics from Telegraf, detects anomalies in CPU, RAM, disk, network, load, processes, and TCP connections, and can create investigations for performance issues.

The second module is **IPS attack visualization**. It displays geo-enriched attacks on a world map, with attack paths, categories, lifecycle status, and real-time events.

The third module is the **AI Assistant**. It allows analysts to ask questions in natural language. The backend extracts keywords, fetches real SOC context, builds a prompt, and sends it to the LLM to generate an answer.

The fourth module is the **AI Operator**. It translates natural language operational requests into Ansible playbooks. For example, an analyst can ask to check RAM, disk, CPU, services, Docker containers, packages, or ports. The system generates a controlled playbook and sends it through the approval workflow.

07 - Functional and Non-Functional Requirements

Slide 24 - Section Introduction

After presenting the architecture and implementation phases, I will now describe the requirements of the platform.

Slide 25 - Functional Requirements

The functional requirements describe what the platform must do.

The platform must collect alerts from different security tools, normalize them, enrich them, and store them. It must correlate alerts into incidents and allow investigations to be created.

It must also provide AI analysis, generate remediation playbooks, support approval workflows, execute actions using Ansible, verify the result, and archive the complete lifecycle.

On the frontend side, the platform must provide dashboards for alerts, incidents, investigations, metrics, IPS visualization, AI assistant, and operational actions.

So functionally, the platform covers the complete SOC chain: detection, analysis, investigation, remediation, verification, and reporting.

Before presenting, make sure the functional requirements table is visible and not empty.

Slide 26 - Non-Functional Requirements

The non-functional requirements describe the quality expected from the system.

For performance, dashboards should respond quickly, and the ingestion pipeline should handle a large number of events.

For security, communications should use TLS, access must be authenticated, actions must be audited, and Elasticsearch must remain internal.

For scalability, the architecture is modular and can support horizontal expansion.

For reliability, the backend uses local persistence, circuit breakers for LLM calls, and graceful error handling.

For maintainability, the platform uses centralized scripts, YAML configuration, and complete logging.

For interoperability, it exposes REST APIs, WebSocket updates, and supports multiple LLM providers.

Correction: replace 'Elasticsearch expose publiquement' with 'Elasticsearch not publicly exposed.'

08 - Testing Strategy

Slide 27 - Section Introduction

In this part, I will explain how the platform is validated and tested.

Slide 28 - Testing Strategy

The testing strategy is divided into several levels.

First, unit tests validate individual components, such as Wazuh, Suricata, and Falco mappers, enrichment logic, incident correlation, Ansible playbook generation, and YAML validation.

Second, integration tests validate the full flow between Elasticsearch, the backend pipeline, SQLite storage, APIs, and WebSocket updates.

Third, functional tests validate realistic scenarios, such as brute-force attacks, port scanning, malware detection, investigation generation, playbook creation, and approval workflow.

Finally, performance and security tests validate event load, dashboard latency, polling stability, firewall rules, TLS, authentication, and audit trails.

The goal is to prove that the platform works technically and operationally.

This slide shows that the project is not only implemented, but also validated.

09 - Methodology

Slide 29 - Section Introduction

Now I will present the methodology followed during the project.

Slide 30 - Development Methodology

The project followed an agile iterative methodology.

The work was divided into three major phases, and each phase produced a functional deliverable.

In Phase 1, the focus was on monitoring foundation: installing and integrating security tools, validating data flow, and building Elasticsearch and Kibana dashboards.

In Phase 2, the focus was on cloud infrastructure: deploying the platform on Huawei Cloud, configuring VPC, security groups, resource optimization, and securing access.

In Phase 3, the focus was on backend intelligence: alert pipeline, incident correlation, AI engine, Ansible remediation, approval workflow, and operational modules.

The project management approach included two-week sprints, continuous code reviews, documentation in parallel, automated tests, and continuous integration of components.

10 - Conclusion

Slide 31 - Section Introduction

In the final part, I will summarize the results achieved and the value of the project.

Slide 32 - Results and Achievements

To conclude, this project delivers a complete intelligent SOC platform.

It provides multi-layer monitoring for network, host, runtime, and infrastructure. It is deployed on Huawei Cloud with a secure architecture based on VPC, security groups, and controlled access.

The backend intelligence layer transforms raw monitoring data into normalized alerts, correlated incidents, investigations, AI analysis, playbooks, approval workflows, execution, verification, and archives.

The project also includes advanced modules such as performance monitoring, IPS attack visualization, AI Assistant, and AI Operator.

The main value of the platform is that it transforms raw security data into actionable SOC intelligence and controlled remediation workflows.

Final Slide - Thank You

Thank you for your attention.

The key idea of this project is that the platform does not only collect and display data. It understands patterns, correlates events, assists analysts, generates remediation actions, and executes them only after validation and approval.

In other words, the platform thinks before acting.

From raw data to intelligent SOC operations.

Strong Sentences to Use During the Presentation

The objective is not only to detect alerts, but to transform them into understandable incidents, investigate them with context, and respond in a controlled and auditable way.

This platform is not just a dashboard; it is a full SOC workflow.

The backend thinks before acting: it validates, waits for approval, executes, verifies, and archives.

Automation supports the analyst, but it does not replace security governance.

Final Checklist Before Presenting

- Change 'Elasticsearch expose publiquement' to 'Elasticsearch not publicly exposed' or 'Elasticsearch remains internal.'
- Change 'Categorys' to 'Categories.'
- Change 'Workflow d'Approval' to 'Approval Workflow.'
- Try to keep the spoken presentation mostly in English, because the slide deck title and structure are in English.
- Check Slide 25: make sure the functional requirements table is visible and not empty.
- During Phase 3, spend more time on the pipeline: raw data -> alerts -> incidents -> investigations -> AI -> playbooks -> approval -> execution -> verification -> archive.

Final closing idea:

The main contribution of this project is building a controlled intelligent SOC platform where automation supports analysts, but never replaces validation and security governance.